Name & Surname:
ID:

CSE 321 - Make-up Quiz

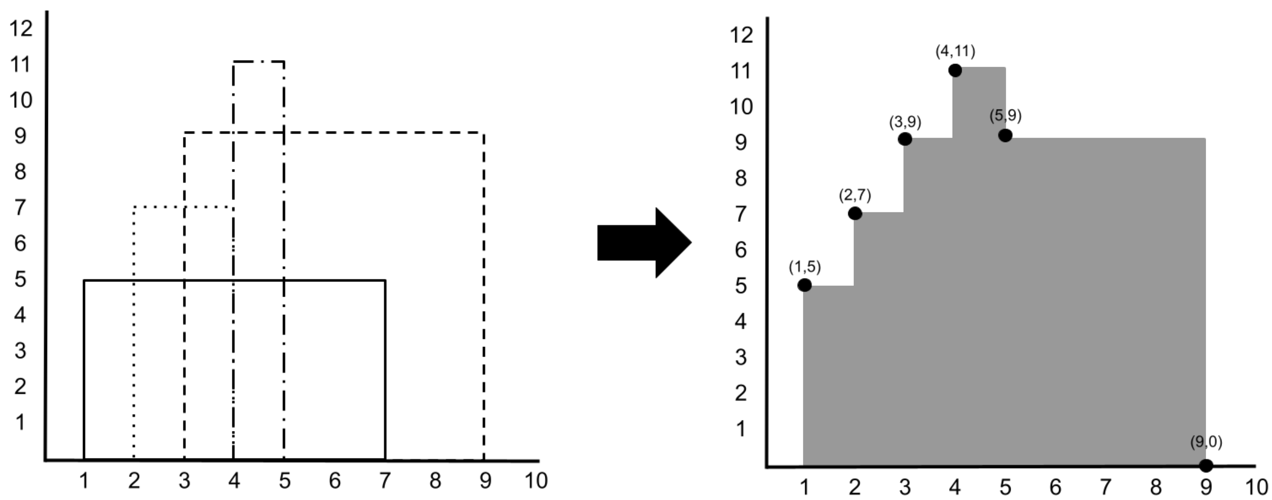
January 3^{rd}, 2025

1. ***30 pts.*** The Skyline Problem involves determining the outline or silhouette of a cityscape formed by overlapping buildings. Each building is represented as a triplet *(left, right, height)*, where left and right are the x-coordinates of the building's edges, and height is its height. The goal is to output the critical points where the skyline changes, forming a series of key points that represent the city's silhouette.

   Design a divide-and-conquer algorithm to solve the Skyline Problem. Provide the pseudo-code of your algorithm along with an explanation, and analyze its worst-case time complexity.

   Example:
   Input: [[1, 7, 5], [2, 4, 7], [3, 9, 9], [4, 5, 11]]
   Output: (1, 5), (2, 7), (3, 9), (4, 11), (5, 9), (9, 0)

   For a better understanding, examine the images below.



2. ***35 pts.*** Consider a sequence of instructions in a program where a list *instructionTypes* represents the types of instructions, and another list *removalCosts* indicates the time required to remove each instruction. To maintain program stability, no two consecutive instructions of the same type can remain in the sequence. The goal is to determine the minimum total cost to remove instructions while satisfying this condition. Design a dynamic programming algorithm to compute the minimum cost. Provide the **recurrence relation** and pseudo-code of your algorithm along with an explanation, and analyze its worst-case time complexity.

Example:
Inputs:
instructionTypes = [LOAD, ADD, ADD, LOAD, STORE, STORE]
removalCosts = [2, 3, 1, 4, 5, 2]

Output: 3

Explanation:
Cost of removing instructionTypes[0] (LOAD) is removalCosts[0] = 2
Cost of removing instructionTypes[1] (ADD) is removalCosts[1] = 3
Cost of removing instructionTypes[2] (ADD) is removalCosts[2] = 1
Cost of removing instructionTypes[3] (LOAD) is removalCosts[3] = 4
Cost of removing instructionTypes[4] (STORE) is removalCosts[4] = 5
Cost of removing instructionTypes[5] (STORE) is removalCosts[5] = 2
In this example, there are two consecutive ADD instructions and two consecutive STORE instructions, requiring one from each pair to be removed. The key is to determine which instruction to remove from each pair. For the ADD instructions, the second one is removed as its removal cost is lower. Similarly, for the STORE instructions, the second one is removed. The total cost for these removals is 1 + 2 = 3.

3. ***35 pts.*** The Set Cover Problem is defined as follows: Given a universe U of elements and a collection of subsets $S_1, S_2, ..., S_n$, the goal is to select the smallest number of these subsets such that their union covers the entire universe U. This means that every element in U must appear in at least one of the chosen subsets.

Design a greedy algorithm to solve the Set Cover problem. Provide the pseudo-code of your algorithm along with an explanation, and analyze its worst-case time complexity.

Example:
Inputs:
U = [1, 2, 3, 4, 5]
S = [[1, 2, 3], [2, 4], [3, 5], [1, 5]]

Output: [[1, 2, 3], [2, 4], [3, 5]] or [[1, 2, 3], [2, 4], [1, 5]]